

Unification and Inference in First-order Logic

Giulia Manara

SDU

09/04/2026

Wumpus World – First-Order Logic

Goal: Represent the Wumpus World in FOL more concisely than propositional logic.

Percepts: Agent receives a vector at each time step:

$\text{Percept}([\text{Stench}, \text{Breeze}, \text{Glitter}, \text{None}, \text{None}], t)$

Percept is a binary predicate, time is an integer.

Actions as terms:

$\text{Turn}(\text{Right}), \text{Turn}(\text{Left}), \text{Forward}, \text{Shoot}, \text{Grab}, \text{Climb}$

Example query:

$\text{ASK}(\text{KB}, \text{BestAction}(a, t)) \Rightarrow \{a / \text{Grab}\}$

Perception Rules – Mapping percepts to facts

Examples of reasoning from percepts:

$$\forall t, s, g, w, c \text{ Percept}([s, \text{Breeze}, g, w, c], t) \Rightarrow \text{Breeze}(t)$$

$$\forall t, s, g, w, c \text{ Percept}([s, \text{None}, g, w, c], t) \Rightarrow \neg \text{Breeze}(t)$$

$$\forall t, s, b, w, c \text{ Percept}([s, b, \text{Glitter}, w, c], t) \Rightarrow \text{Glitter}(t)$$

$$\forall t, s, b, w, c \text{ Percept}([s, b, \text{None}, w, c], t) \Rightarrow \neg \text{Glitter}(t)$$

Key idea: Quantification over time avoids repeating axioms for each step.

Simple Reflex Rules – Best Action

$$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$$

Example:

$$\text{Glitter}(5) \Rightarrow \text{BestAction}(\text{Grab}, 5)$$

Objects and Locations

Squares: Represented as lists $[row, col]$ instead of naming each.

Example adjacency:

$$\forall x, y, a, b \text{ Adjacent}([x,y],[a,b]) \Leftrightarrow$$

$$(x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

Pits and Wumpus:

$$\text{Pit}([x,y]), \quad \text{Wumpus}(\text{constant})$$

Agent location:

$$\text{At}(\text{Agent}, [1,3], t), \quad \text{At}(\text{Wumpus}, [1,3], t)$$

Example: Breezy squares from percept:

$$\forall x, y, t \text{ At}(\text{Agent}, [x, y], t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}([x, y])$$

Pits from breezy info:

$$\forall x, y \text{ Breezy}([x, y]) \Leftrightarrow \exists a, b \text{ Adjacent}([a, b], [x, y]) \wedge \text{Pit}([a, b])$$

What we covered so far:

- **Propositional logic:** models, entailment, equivalences, contradiction-based proof
- **Resolution for PL:** rule, CNF conversion, algorithm, closure/termination, completeness intuition
- **Efficient PL inference:** model-checking improvements and DPLL optimization ideas
- **FOL foundations:** syntax, semantics, quantifiers, models, and truth conditions
- **Knowledge representation:** natural numbers and Wumpus World in FOL

Inference in FOL - chapter 9

From First-Order to Propositional Inference

Key idea: Turn a FOL formula into an equivalent propositional formula, then apply propositional inference.

Propositionalization:

- 1 Replace universally quantified variables with concrete objects (Universal Instantiation)
- 2 Replace existentially quantified variables with new names (Existential Instantiation)
- 3 Transform the resulting sentences into propositional form

Advantage: we can reuse already known, complete propositional inference algorithms.

Notation

Substitution: A mapping from variables to terms, denoted as $\theta = \{x_1/t_1, x_2/t_2, \dots\}$.

Applying a substitution: $\text{Subst}(\theta, \phi)$ means replacing each variable x_i in ϕ with the corresponding term t_i .

Example: $\theta = \{x/\text{John}\}$ and $\phi = \text{King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x)$

$$\text{Subst}(\theta, \phi) = \text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John})$$

Literal: atomic sentence or its negation

$$l_1 : \text{King}(\text{John}), l_2 : \text{Greedy}(\text{John})$$

$$m_1 : \neg \text{King}(\text{John}), m_2 : \neg \text{Greedy}(\text{John})$$

1: Universal Instantiation (UI) - A new inference rule

$$UI \frac{\forall x \alpha}{\text{Subst}(\{x/t\}, \alpha)} t \text{ term}$$

Idea: replace the variable x with a concrete term t .

$$\alpha = \forall x \text{ King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x)$$

$$\theta_1 = \{x/\text{John}\} \quad \theta_2 = \{x/\text{Richard}\}$$

$$\text{Subst}(\theta_1, \alpha) = \text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John})$$

$$\text{Subst}(\theta_2, \alpha) = \text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \rightarrow \text{Evil}(\text{Richard})$$

Note: Universal instantiation is **sound**.

2: Existential Instantiation (EI) - A new inference rule

$$EI \frac{\exists x \alpha}{\text{Subst}(\{x/k\}, \alpha)} \text{ } k \text{ new constant}$$

Idea: replace the variable x with a **new** constant k .

$$\alpha = \exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

$$\theta = \{x/C1\}$$

$$\text{Subst}(\theta, \alpha) = \text{Crown}(C1) \wedge \text{OnHead}(C1, \text{John})$$

Note: Universal instantiation is **sound**.

About the EI side condition: " k new constant "

An existential statement says that *something exists* $\Rightarrow$ we give it a **name**

$$\exists x P(x) \quad \Rightarrow \quad P(k)$$

k must be a **new name** (never used before)

Analogy:

$$\exists x \left(x > 2.71828 \wedge \frac{d(x^y)}{dy} = x^y \right) \Rightarrow e > 2.71828 \wedge \frac{d(e^y)}{dy} = e^y$$

I don't call it π .

In logic, this new name is called a **Skolem constant**.

Universal Instantiation (UI): many sentences

$$\forall x \text{ Kill}(x, \text{Victim})$$

$$\Rightarrow \begin{array}{ccc} x \rightarrow \text{Alice} & , & x \rightarrow \text{Bob} & , & x \rightarrow \text{Carol} & \dots \\ \text{Kill}(\text{Alice}, \text{Victim}) & , & \text{Kill}(\text{Bob}, \text{Victim}) & , & \text{Kill}(\text{Carol}, \text{Victim}) & \dots \end{array}$$

Existential Instantiation (EI): only one sentence

$$\exists x \text{ Kill}(x, \text{Victim}) \Rightarrow \text{Kill}(\text{Murderer}, \text{Victim})$$

UI $\Rightarrow$ many instances EI $\Rightarrow$ just one new constant

3: Propositional variables

Replace atomic sentences with propositional symbols.

Example:

$King(\text{John}) \wedge Greedy(\text{John}) \rightarrow Evil(\text{John})$ becomes

$JohnIsKing \wedge JohnIsGreedy \rightarrow JohnIsEvil$

Example of Propositionalization

Knowledge base:

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x)$$

$$\text{King}(\text{John}), \quad \text{Greedy}(\text{John}), \quad \text{Brother}(\text{Richard}, \text{John})$$

Step 1: Apply UI with all objects:

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John})$$

$$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \rightarrow \text{Evil}(\text{Richard})$$

Example of Propositionalization

Step 3: Replace with propositional symbols:

$\text{JohnIsKing} \wedge \text{JohnIsGreedy} \rightarrow \text{JohnIsEvil}$

$\text{RichardIsKing} \wedge \text{RichardIsGreedy} \rightarrow \text{RichardIsEvil}$

$\text{JohnIsKing}, \text{JohnIsGreedy}, \text{RichardIsBrotherOfJohn}$

Apply propositional reasoning:

JohnIsEvil (equivalent to $\text{Evil}(\text{John})$)

Propositionalization and Infinite Terms

Problem: Function symbols can generate infinitely many ground terms.

Example: Using the function Father:

John, Father(John), Father(Father(John)), ...

Consequence: Naive propositionalization leads to an infinite set of ground instances.

Herbrand's Theorem

Herbrand (1930): If a sentence is entailed by a first-order KB, there exists a **finite subset** of the propositionalized KB sufficient for a proof.

Procedure:

- Generate all instantiations with constants (John, Richard)
- Generate terms of depth 1 (Father(John), Father(Richard))
- Generate terms of depth 2 (Father(Father(John)), ...), 3, ...
- Continue until proof is found

Result: Completeness is guaranteed even if the full space of ground terms is infinite.

Semidecidability of Propositionalization

Problem: We cannot know in advance if a sentence is not entailed.

Reason: First-order proof search may generate **infinitely nested terms**, possibly never finishing.

First-order entailment is **semidecidable**:

- Algorithms exist that say **YES** if a sentence is entailed
- No algorithm can always say **NO** if it is not entailed

Analogy: Turing & Church (1936), Turing Machine **halting problem**.

Unification

Why Unification?

Avoid useless instantiations.

Knowledge Base:

$$\forall x (\text{King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x))$$

$$\text{King}(\textit{John}) \quad \text{Greedy}(\textit{John})$$

Query: AskVars(Evil(x))

Idea: Only the right substitution is needed: $\{x/\textit{John}\}$

Generalized Modus Ponens

$$\frac{p'_1 \quad \dots \quad p'_n \quad p_1 \wedge \dots \wedge p_n \Rightarrow q}{\text{SUBST}(\theta, q)} \text{ GMP}$$

if $\text{SUBST}(\theta, p'_i) = \text{SUBST}(\theta, p_i)$, for all i

Example:

$$\frac{\text{King}(\text{John}) \quad \text{Greedy}(y) \quad \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)}{\text{Evil}(\text{John})} \text{ GMP}$$

$$\theta = \{x/\text{John}, y/\text{John}\}$$

Example of use of GMP

Knowledge base:

$$\text{Parent}(x, y) \wedge \text{Parent}(y, z) \rightarrow \text{Grandparent}(x, z)$$

$$\text{Parent}(\textit{John}, \textit{Mary}) \quad \text{Parent}(\textit{Mary}, \textit{Alice})$$

Substitution:

$$\theta = \{x/\textit{John}, y/\textit{Mary}, z/\textit{Alice}\}$$

Inference:

$$\frac{\text{Parent}(\textit{John}, \textit{Mary}) \quad \text{Parent}(\textit{Mary}, \textit{Alice}) \quad \text{Parent}(x, y) \wedge \text{Parent}(y, z) \rightarrow \text{Grandparent}(x, z)}{\text{Grandparent}(\textit{John}, \textit{Alice})}$$

Example of use of GMP

Knowledge base:

$$> (x, y) \wedge > (y, z) \rightarrow > (x, z)$$

$$> (3, 2) \quad > (2, 1)$$

Substitution:

$$\theta = \{x/3, y/2, z/1\}$$

Inference:

$$\frac{\begin{array}{ccc} > (3, 2) & > (2, 1) & > (x, y) \wedge > (y, z) \rightarrow > (x, z) \end{array}}{> (3, 1)}$$

Unification in First-Order Logic

Unification is the process of finding substitutions that make two logical sentences identical.

Definition

A **unifier** of two sentences p and q is a substitution θ such that when applied to both sentences, they become identical:

UNIFY is an **algorithm** such that:

$$\text{UNIFY}(p, q) = \theta \quad \text{where} \quad \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

Examples of Unification

Examples:

- **UNIFY**(Knows(John, Jane), Knows(John, x)) $\rightarrow \{x/\text{Jane}\}$
- **UNIFY**(Knows(y, Bill), Knows(John, x)) $\rightarrow \{x/\text{Bill}, y/\text{John}\}$
- **UNIFY**(Knows(y, Mother(y)), Knows(John, x)) $\rightarrow \{y/\text{John}, x/\text{Mother(John)}\}$
- **UNIFY**(Knows(Elizabeth, x), Knows(John, x)) $\rightarrow$ failure
- **UNIFY**(Knows(x, Elizabeth), Knows(John, x)) $\rightarrow$ failure (x conflict)

Unification works as long as a consistent substitution exists for all variables.

Standardizing Apart

Problem: Conflicting variable names can cause unification to **fail**:

UNIFY(Knows(*x*, Elizabeth), Knows(John, *x*)) $\rightarrow$ failure (*x* conflict)

Solution: Rename variables in one sentence to avoid clashes.

Example:

Knows(*x*, Elizabeth) $\rightarrow$ Knows(*y*, Elizabeth)

Now unification with Knows(John, *x*) succeeds:

UNIFY(Knows(John, *x*), Knows(*y*, Elizabeth)) = $\{x/Elizabeth, y/John\}$

Most General Unifier (MGU)

Problem: Some expression pairs have multiple unifiers.

Example:

UNIFY(Knows(John, x), Knows(y, z))

Possible unifiers:

- $\theta_1 = \{y/\text{John}, x/z\}$ (MGU)
- $\theta_2 = \{y/\text{John}, x/\text{John}, z/\text{John}\}$

From general to specific:

$$\theta_2 = \theta_1 \circ \{z/\text{John}\}$$

MGU places the fewest restrictions.

Other unifiers can be derived from the MGU by further substitutions.

When unifying a variable with a complex term, the variable cannot occur inside that term.

Example: $S(x)$ cannot unify with $S(S(x))$

Reason: Otherwise, no consistent substitution exists.

Note: Some systems skip this check for performance, leaving the user responsible for avoiding unsound inferences.

Unification Algorithm

function UNIFY($x, y, \theta = \text{empty}$) **returns** a substitution to make x and y identical, or *failure*
 if $\theta = \text{failure}$ **then return** *failure*
 else if $x = y$ **then return** θ
 else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)
 else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)
 else if COMPOUND?(x) **and** COMPOUND?(y) **then**
 return UNIFY(ARGS(x), ARGS(y), UNIFY(OP(x), OP(y), θ))
 else if LIST?(x) **and** LIST?(y) **then**
 return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y), θ))
 else return *failure*

function UNIFY-VAR(var, x, θ) **returns** a substitution
 if $\{var/val\} \in \theta$ for some val **then return** UNIFY(val, x, θ)
 else if $\{x/val\} \in \theta$ for some val **then return** UNIFY(var, val, θ)
 else if OCCUR-CHECK?(var, x) **then return** *failure*
 else return add $\{var/x\}$ to θ

UNIFY Example

Goal: Run

$\text{UNIFY}(\text{Knows}(y, \text{John}), \text{Knows}(\text{Bill}, x))$

Step 1: Compare top-level compound terms

Both have predicate Knows, so we recursively unify arguments:

$\text{UNIFY}((y, \text{John}), (\text{Bill}, x), \theta = \emptyset)$

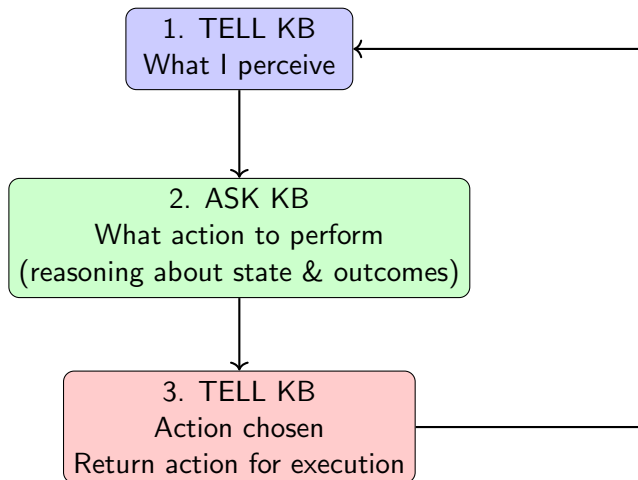
Step 2: First arguments

$\text{UNIFY}(y, \text{Bill}, \theta = \emptyset) = \{y/\text{Bill}\}$

Step 3: Second arguments

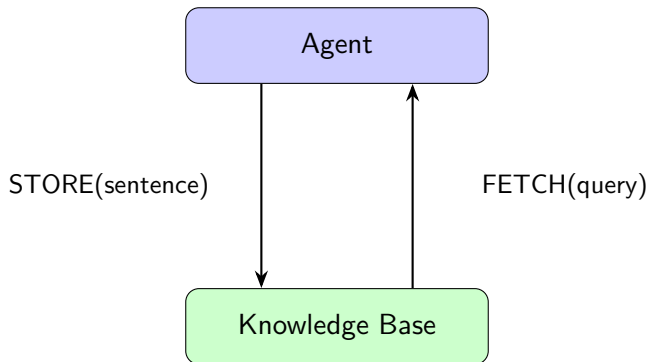
$\text{UNIFY}(\text{John}, x, \{y/\text{Bill}\}) = \{y/\text{Bill}, x/\text{John}\}$

ASK and TELL



STORE and FETCH

ASK and TELL are instances of the more primitives **STORE** and **FETCH**:



FETCH(query) returns **all unifiers** such that the query unifies with **some sentence in the knowledge base**.

Efficient Implementation of STORE and FETCH

Naive approach: Every query is unified against **every fact**.

Pros: Simple, correct.

Cons: Very inefficient.

Knows(**John**, **x**)

would be tried against all facts, even unrelated ones such as

Brother(**Richard**, **John**)

Efficient Implementation of STORE and FETCH

Optimization: Predicate Indexing

- Facts are grouped by **predicate symbol**.
- Each predicate has its **bucket** of facts.
- Queries only attempt unification with facts in the relevant bucket.

Example:

- Bucket Knows: all Knows facts
- Bucket Brother: all Brother facts
- Query Knows(John, x) only checks the Knows bucket

Predicate indexing is **very efficient** when each predicate has **few facts**.

Indexing Facts for Efficient Fetch

Example: Tax authorities want to keep track of who employs whom, using a predicate `Employs(x, y)`. **Too large of a bucket.**

Idea: Store the fact under multiple index keys for fast access to all relevant queries. Then retrieve the key from the query and only check the relevant facts.

`Employs(IBM, y)` indexed under "Employs" and "*IBM*".

Subsumption Lattice

Given a sentence to be stored, construct indices for all possible queries that unify with it.

Queries form **subsumption lattice**:

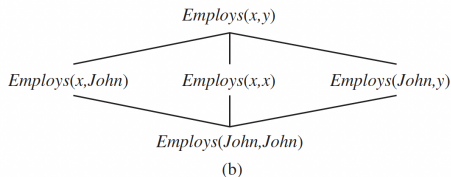
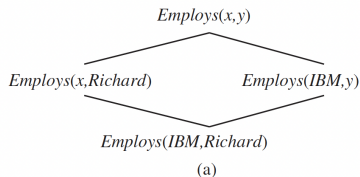


Figure 9.2 (a) The subsumption lattice whose lowest node is *Employes(IBM, Richard)*. (b) The subsumption lattice for the sentence *Employes(John, John)*.

Properties:

- Child nodes are obtained from parent by a **single substitution**.
- Highest common descendant of two nodes = **most general unifier**.

Storage cost vs retrieval speed

For predicates with n arguments, lattice has $O(2^n)$ nodes.

Solution: Limit the indexes.

- Fixed policy, s.a. indexing only predicate plus a single argument
- Learn an adaptive policy based on frequent queries.

Resolution

Recap: Propositional Resolution

Return true if $KB \models \alpha$ false otherwise.

Steps:

- 1 Take conjunction of the KB and the negation of the query:

$$KB \wedge \neg \alpha$$

- 2 Convert all sentences to **Conjunctive Normal Form (CNF)**:

$$\text{CNF}(KB \wedge \neg \alpha)$$

- 3 Apply the **resolution rule** on all pairs of clauses:

$$\frac{C_i \quad C_j}{\text{resolvent}}$$

- 4 Repeat until:

- **Empty clause** ($\square$) is derived $\rightarrow \alpha$ is **entailed** (True)
- No new resolvents $\rightarrow \alpha$ is **not entailed** (False)

Why is it interesting?

Resolution is **general**: works for **any knowledge base**, not just definite clauses.

It extends to first-order logic (FOL).

CNF Conversion: Step 1 - Eliminate Implications

Original sentence:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$$

Step 1: Replace $\Rightarrow$ with $\vee$ and negate antecedents:

$$\forall x \neg [\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$$

CNF Conversion: Step 2 - Move Negation Inwards

Negation Rules:

- $\neg \forall x \, p \Rightarrow \exists x \, \neg p$
- $\neg \exists x \, p \Rightarrow \forall x \, \neg p$

Sentence transformations:

$$\forall x \, [\exists y \, \neg(\neg \text{Animal}(y) \vee \text{Loves}(x, y))] \vee [\exists y \, \text{Loves}(y, x)]$$

$$\forall x \, [\exists y \, (\neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x, y))] \vee [\exists y \, \text{Loves}(y, x)]$$

$$\forall x \, [\exists y \, (\text{Animal}(y) \wedge \neg \text{Loves}(x, y))] \vee [\exists z \, \text{Loves}(z, x)]$$

CNF Conversion: Step 3 - Skolemization

Naively applying existential instantiation:

$$\forall x [\text{Animal}(A) \wedge \neg \text{Loves}(x, A)] \vee \text{Loves}(B, x)$$

Issue: Meaning changes! Now "everyone either fails to love the same animal A or is loved by the same person B ".

Skolemization: For existential variables in the scope of a $\forall$:

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$$

- $F(x)$ = an animal potentially unloved by x
- $G(x)$ = someone who might love x
- Skolem functions ensure correct semantics

CNF Conversion: Step 4 - Distribute OR over AND

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)]$$

Result: Sentence is now in CNF, ready for resolution.

Resolution Rule in FOL

$$RR^1 \frac{l_1 \vee \dots \vee \ell_i \vee \dots \vee l_k \quad m_1 \vee \dots \vee m_j \vee \dots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)} \theta$$

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$

Resolution Examples

Example 1

$$\text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \quad \neg\text{Loves}(u, v) \vee \neg\text{Kills}(u, v)$$

$$\theta = \{u/G(x), v/x\}$$

$$RR^1 \frac{\text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \quad \neg\text{Loves}(u, v) \vee \neg\text{Kills}(u, v)}{\text{Animal}(F(x)) \vee \neg\text{Kills}(G(x), x)} \theta$$

Example 2

$$P(x) \vee Q(x) \quad \neg P(a)$$

$$\theta = \{x/a\}$$

$$RR^1 \frac{P(x) \vee Q(x) \quad \neg P(a)}{Q(a)} \theta$$

Resolution in FOL is incomplete

Clauses:

$$\forall x \forall y (P(x) \vee P(y)) \quad \forall x \forall y (\neg P(x) \vee \neg P(y))$$

Clearly **unsatisfiable**, but resolution alone cannot derive the empty clause.

CNF clauses:

$$P(x) \vee P(y) \quad \neg P(u) \vee \neg P(v)$$

Resolution + Factoring is Complete

Binary resolution attempt:

$$\theta = \{x/u\}$$

$$\frac{P(x) \vee P(y) \quad \neg P(u) \vee \neg P(v)}{P(y) \vee \neg P(v)}$$

Empty clause will never be derived!

Resolution + Factoring is Complete

Factoring:

$$P(x) \vee P(y) \xrightarrow{\text{fact}} P(y)$$

$$\neg P(u) \vee \neg P(v) \xrightarrow{\text{fact}} \neg P(u)$$

Resolution after factoring:

$$\theta = \{y/u\}$$

$$RR^1 \frac{P(y) \quad \neg P(u)}{\square}$$

Factoring and Completeness

- **Binary resolution:** resolves exactly two complementary literals at a time
- **Factoring:** remove redundant literals
 - Propositional: identical literals
 - First-order: unifiable literals (apply unifier to entire clause)
- **Complete inference:** combination of binary resolution + factoring

Recap: Resolution in First-Order Logic

Return true if $KB \models \alpha$ false otherwise.

Steps:

- Take the knowledge base and the negation of the query:

$$KB \wedge \neg \alpha$$

- Convert to **CNF**:

-

Eliminate $\rightarrow$, move $\neg$, standardize variables

- **Skolemization** (remove $\exists$):

introduce **Skolem constants / functions**

- Drop universal quantifiers and obtain clauses:

$$\text{CNF}(KB \wedge \neg \alpha)$$

Recap: Resolution in First-Order Logic

- Apply **resolution + factoring**
- Repeat until:
 - $\square$ derived $\Rightarrow$ **entailed**
 - no new clauses $\Rightarrow$ **not entailed**

Key Strategies for Efficient FOL Resolution

Unit Preference: prefer unit clauses to shorten resolvents Complete for Horn clauses, resembles forward chaining

Set-of-Support:

- Resolve with at least one clause from the set of support
- completeness: choose the set of support so that the remainder of the sentences are jointly satisfiable.

Input / Linear Resolution:

- Input: always use one clause from the KB/query, complete for Horn KB
- Linear: allow resolution with ancestor clauses, complete for general KBs

Subsumption and Learning

Subsumption:

- Discard sentences **subsumed by existing clauses** (more specific)
- Keeps KB small → reduces search space

Learning:

- Use previously proved theorems to guide resolution
- Train ML models to **predict successful proof steps**
- Example: **DEEPHOL** uses deep embeddings of goals and premises

Practical Uses of FOL Resolution

Applications:

- **Hardware verification:** prove CPU correctness, timing properties
- **Software verification:** prove correctness of programs (SPIN, Remote Agent)
- **Deductive synthesis:** constructively prove existence of programs satisfying specifications
- **Cryptography and algorithms:** verify RSA, Boyer–Moore string matching

Note: Most effective in **formally defined domains**.

Forward Chaining

Forward Chaining in Propositional Logic

Horn Clause:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \rightarrow Q$$

Conjunction of positive literals implies a single positive literal

Forward Chaining Algorithm:

- ① Start with KB (all known facts)
- ② Apply inference rules to derive new facts
- ③ Repeat until:
 - Query α is derived $\rightarrow$ stop, return true
 - No new facts can be derived $\rightarrow$ stop, return false

Properties: Sound and Complete for Horn clauses

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

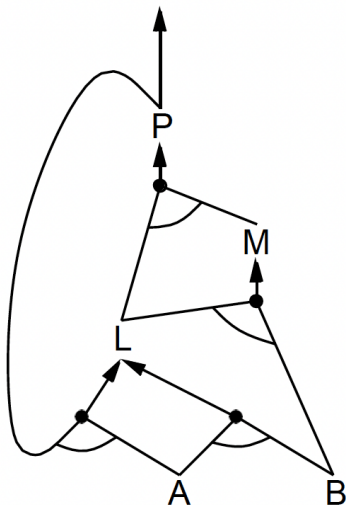
$$B \wedge L \Rightarrow M$$

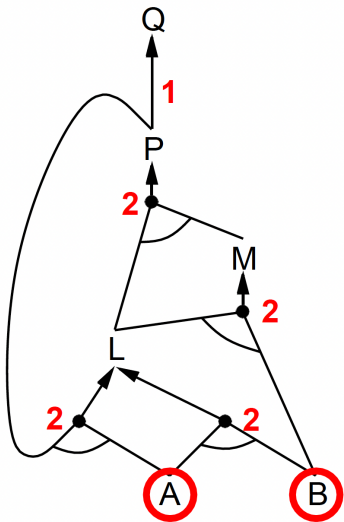
$$A \wedge P \Rightarrow L$$

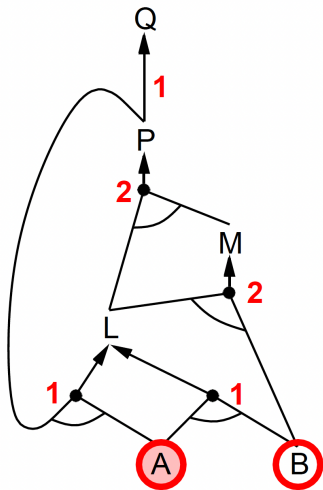
$$A \wedge B \Rightarrow L$$

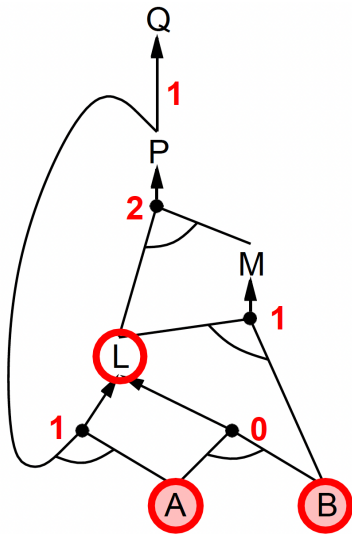
A

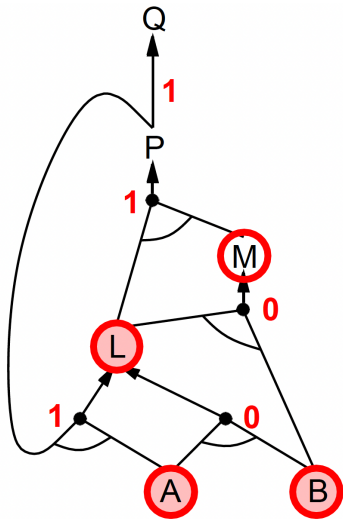
B

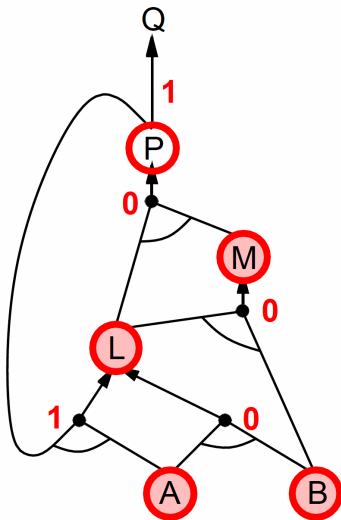


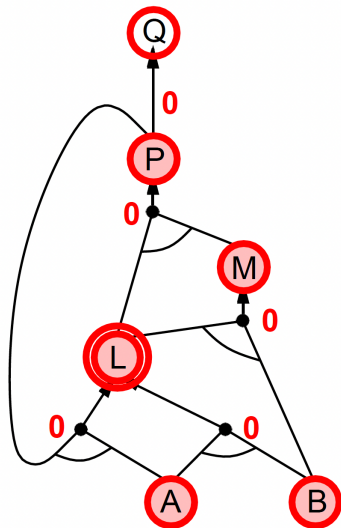


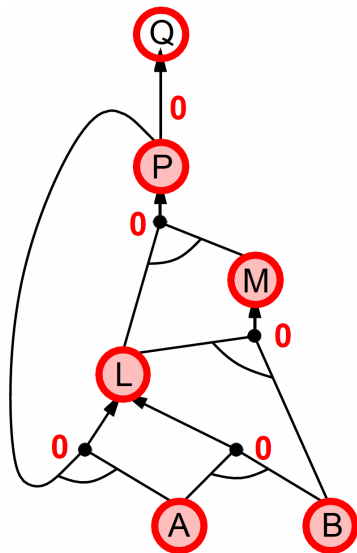


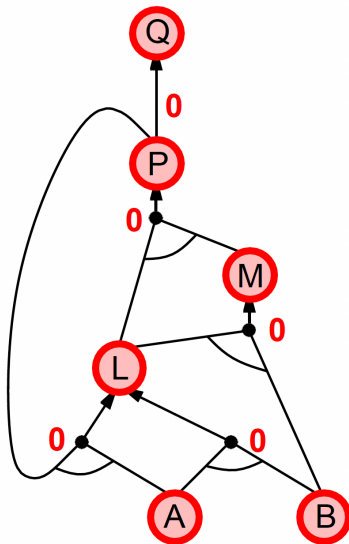












First-Order Definite Clauses

Definition: A **first-order definite clause** is a **disjunction of literals** with **exactly one positive literal**.

Form:

- Atomic clause: *King(John)*
- Implication: *King(x) ∧ Greedy(x) → Evil(x)*

Notes:

- Existential quantifiers are not allowed
- Universal quantifiers are implicit (e.g., *Greedy(y)* means $\forall y \text{ Greedy}(y)$)
- Literals can include variables

FOL Forward Chaining: Idea

Input: KB of first-order definite clauses, query α

Process:

- 1 Start with known facts in **KB**.
- 2 Repeatedly apply **Generalized Modus Ponens**:

$$\frac{\text{premises satisfied in KB}}{\text{conclusion}}$$

- 3 Add new inferred facts to KB.
- 4 Stop when:
 - **Query is derived**, or
 - **No new facts** can be added.

Note: Facts that are **renamings** of known facts (e.g., Likes(x,IceCream) vs Likes(y,IceCream)) are **not new**.

FOL Forward Chaining: Soundness & Termination

Soundness: All inferred facts come from **Generalized Modus Ponens** $\rightarrow$ correct.

Completeness: For **definite clause KBs**, will derive any entailed query.

With function symbols:

- Function symbols can produce **infinitely many facts** $\rightarrow$ algorithm may not terminate.

Example: Peano axioms generate $\text{NatNum}(S(0))$, $\text{NatNum}(S(S(0)))$,
... infinitely.

- Semi-decidable: algorithm can confirm entailment if it exists, but may run forever if query not entailed.

Summary - Part II

From chapter 9 to the end:

- **FOL inference bridge**: propositionalization with UI/EI and their side conditions
- **Unification**: substitutions, MGU, standardizing apart, occur-check, and UNIFY steps
- **FOL resolution pipeline**: CNF conversion, resolution rule, examples, and why factoring is needed
- **Practical proof control**: factoring, subsumption, linear/input strategies, and learning support
- **FOL Forward Chaining**: Horn clauses, algorithm, examples, and properties

Reading exercise: section 9.5.3 for examples of resolution in practice.